Fast Unbalanced Private Computation on Set Intersection from Permuted Multi-Query Private Membership Test

Weizhan Jing[1,2], Xiaojun Chen[1,2(✉)], Xudong Chen[2], Ye Dong[3], Yaxi Yang[4] and Qiang Liu[1,2]

[1] School of Cyber Security, University of Chinese Academy of Sciences
[2] Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences
{jingweizhan,chenxiaojun,chenxudong,liuqiang2022}@iie.ac.cn
[3] School of Computing, National University of Singapore
{dongye}@nus.edu.sg
[4] iTrust, Singapore University of Technology and Design
{yxyangjnu}@gmail.com

Abstract. Unbalanced private computation on set intersection (uPCSI) enables two parties to securely compute fine-grained functions over the intersection of X and Y, where $|Y| \ll |X|$. Existing works proposed a uPCSI framework based on fully homomorphic encryption (FHE)-based private set intersection (PSI) protocols. However, their solutions face efficiency limitations, as they introduce an additional comparison procedure with a complexity of $\mathcal{O}(|Y| \log |X|)$.

In this paper, we present a lightweight uPCSI framework with semi-honest security. First, we propose a permuted multi-query private membership test (pmqPMT) protocol and its labeled variant from the FHE-based PSI, thereby avoiding the costly comparison procedure. Upon our pmqPMT, we propose an optimized uPCSI framework for computing arbitrary functions over the intersection, along with several specific optimizations for better efficiency. Besides, our framework can be extended to support more comprehensive labeled uPCSI requirements, covering both single-labeled and double-labeled cases. Compared to the state-of-the-art uPCSI protocols, we achieve over a 4.7× online speedup and reduce communication costs by 15% on average.

Keywords: Privacy · Unbalanced Private Computation on Set Intersection · Permuted mqPMT

1 Introduction

Private set intersection (PSI) [7, 10, 14, 34, 42] is a cryptographic primitive that enables two parties (i.e., a receiver with a set Y and a sender with a set X) to securely compute the intersection $X \cap Y$ without revealing any extra information. Many practical applications [3, 12, 23, 37, 41] involve PSI, but most of them

focus on obtaining some predefined functions over the intersection, rather than the items in the intersection set [14, 42]. An example is illustrated in Figure 1. In the advertising conversion evaluation, the receiver (merchant) holds a list of transactions, which includes the user ID and the cost, while the sender (advertiser) maintains information about which user clicked on specific advertisements during their web sessions. Both parties hope to learn i) how many users clicked ads and made purchases, and ii) the total cost of those users, without disclosing their respective lists. This motivates the study of private computation on set intersection (PCSI) protocols, which extend PSI by enabling secure computation of functions on the intersection.

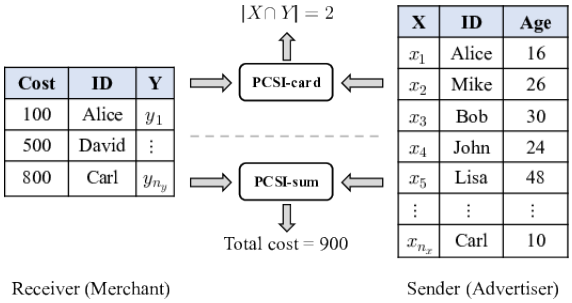

Fig. 1. PCSI in advertising conversion evaluation.

Earlier works of PCSI primarily support particular functions (i.e., intersection cardinality [12,15,22,41] and sum [23]). However, these specialized solutions are difficult to extend for general-purpose function computation.

Then, the works of [8,18] propose the general PCSI to support arbitrary function computation. These approaches first construct a multi-query reverse private membership test (mqRPMT) protocol to compute the intersection set and then feed the intersection set into a generic 2PC protocol to evaluate predefined functions, including particular and general functions. Chen *et al.* [8] achieves linear communication complexity proportional to the size of both sets, which is the same as the protocols of particular functions. However, in the unbalanced setting, where one set is significantly smaller than the other, the communication overhead of [8] becomes more expensive compared to the protocols of particular functions [15, 37, 41], which have communication linear in the size of the smaller set and logarithmic to the size of the larger set.

To alleviate this problem, Tu *et al.* [40] introduce an unbalanced PCSI (uPCSI) framework with communication complexity that scales linearly with the smaller set size and logarithmically with the larger set size. Their work [40] constructs a permuted multi-query private membership test (pmqPMT) protocol, instead of the mqRPMT, to reduce communication. The pmqPMT construction of [40] is directly based on the fully homomorphic encryption (FHE)-based PSI but requires an additional permuted matrix private equality test (pmPEQT) procedure, which is of $\mathcal{O}(n_y \log n_x)$ computational and communication complexity. However, [40] still has an efficiency gap compared to the existing specific uPCSI protocol for the particular function. For the simplest case of semi-honest

uPCSI, i.e., uPCSI-card, the protocol of [40] is 6.1× slower and requires 1.3× more communication than the state-of-the-art (SOTA) protocol [41]. *Therefore, the first challenge is designing a more efficient uPCSI protocol.*

In practical data collaboration scenarios, both parties often hold sensitive labels linked to their identifiers. A typical example is the advertising conversion evaluation scenario illustrated in Figure 1. Specifically, the sender (advertiser) possesses user identifiers with sensitive user attribute labels (e.g., age); the receiver (merchant) holds transaction records including user identifiers and sensitive transaction labels (e.g., transaction amount). In this scenario, the two parties seek to compute the proportion of the total transaction amount attributable to users aged 20 to 30, thereby evaluating the appeal of products to the target age group. This scenario precisely constitutes a double-labeled uPCSI problem.

Existing uPCSI solutions [40] are restricted to single-labeled inputs, where attributes reside on only one party. Extending the approach of [40] to the double-labeled setting is non-trivial. The core difficulty lies in the oblivious alignment of private inputs. In efficient uPCSI protocols, intersection results are typically permuted using a secret permutation known only to the sender to conceal access patterns. As a result, the receiver cannot align its private labels with the permuted results without either learning the permutation (violating sender privacy) or revealing its labels (violating receiver privacy). Addressing this challenge requires an efficient mechanism for oblivious permutation or share translation, which is absent in the single-labeled setting. *Therefore, the second challenge is to design an efficient uPCSI protocol that supports double-labeled inputs.*

Our Contributions. To address the above shortcomings, we propose a lightweight pmqPMT protocol along with its labeled variant, and develop efficient uPCSI frameworks upon them as follows:

- **Lightweight pmqPMT (with label):** Our first key insight is that the oblivious pseudorandom function (OPRF) of the unbalanced PSI protocol [6, 10] can be seamlessly substituted by the permuted OPRF (pOPRF). This insight allows us to bypass the computationally expensive pmPEQT procedure [40]. Based on this substitution, we construct a lightweight pmqPMT with better efficiency, and extend it to support labeled cases. Compared to [40], our pmqPMT improves the online runtime by approximately 5.1× in LAN and saves 23% communication costs on average.
- **Efficient uPCSI Framework:** We design an efficient uPCSI framework on top of our lightweight pmqPMT to support various types of computation, including card, card-sum, and secret-sharing based general functions. Compared to SOTA work [40], our framework reduces communication overhead by 15% and improves online runtime by 4.7× on average.
- **Extension to Double-labeled:** Benefiting from our pmqPMT with labels, we extend our uPCSI to cover not only single-labeled but also double-labeled cases efficiently. The communication complexity of our solutions is linear with respect to the size of the smaller set and logarithmic to the size of the larger set. To our best knowledge, it is the first uPCSI to support secret sharing-based general functions computation in the double-labeled case.

2 Preliminary

2.1 Notation

The sender ($\mathcal{S}$) and the receiver ($\mathcal{R}$) are two parties in PSI, and their respective private sets are X and Y. Let $\mathbb{F}$ be a finite field with elements of ℓ bits. $[n]$ denotes a set $\{1, ..., n\}$. A lowercase bold letter $\boldsymbol{a}$ denotes a vector and a_i is the i-th element. $[\![\cdot]\!]$ indicates the ciphertext. A permutation π over $[n]$ means that $\pi : [n] \to [n]$ is a bijective function. We write $\pi(\{a_1, ..., a_n\})$ as $\{a_{\pi(1)}, ..., a_{\pi(n)}\}$, where $a_{\pi(i)}$ indicates the i-th item after the permutation. We use $\langle x \rangle_1$ and $\langle x \rangle_0$ to denote shares of an item $x \in \mathbb{F}$ between the $\mathcal{S}$ and $\mathcal{R}$. $F_k(\cdot)$ is an OPRF function with key k. The computational and statistical security parameters are denoted as κ, λ, respectively.

2.2 Cryptographic Techniques

Permuted OPRF. The OPRF [17] enables $\mathcal{R}$ to get the $F_k(y)$ without revealing its y to $\mathcal{S}$, and $\mathcal{S}$ keeps its secret key k privately as well. Permuted OPRF (pOPRF) [8] is an extension of OPRF, which enables permutation operation: $\mathcal{S}$ additionally chooses a random permutation π over $[n]$, and $\mathcal{R}$ learns its OPRF values in permuted order, namely, $y'_i = F_k(y_{\pi(i)})$. In this paper, we use the Diffie-Hellman (DH)-based pOPRF [8].

Oblivious Transfer. Oblivious transfer (OT) [33] is a fundamental primitive in the two-party secure computation. In the 1-out-of-2 OT, $\mathcal{S}$ holds two messages (m_0, m_1), and $\mathcal{R}$, with a choice bit $b \in \{0, 1\}$, learns only m_b without revealing b or learning anything about m_{1-b}. Early OT protocols relied on costly public-key operations. Ishai *et al.* [24] introduced OT extension, enabling a large number of OTs using only a small number of public-key operations. The formal functionality is given for reference in Appendix A.

Additive Secret Sharing. We use the additive secret sharing [13] scheme over the field $\mathbb{F}$ as the underlying framework for our protocol and use $\langle x \rangle = (\langle x \rangle_0, \langle x \rangle_1)$ to denote the 2-out-of-2 additive secret shares of an item $x \in \mathbb{F}$. Two parties, $\mathcal{R}$ and $\mathcal{S}$, hold $\langle x \rangle_0, \langle x \rangle_1$, respectively. Shares are generated by sampling random elements $\langle x \rangle_0$ and $\langle x \rangle_1$ from $\mathbb{F}$ with the constraint that $\langle x \rangle_0 + \langle x \rangle_1 = x$. Addition is trivial to see. The multiplication functionality $\mathcal{F}_{\text{MULT}}$ can be realized using beaver triples [1].

Share Translation. Share translation (ST) [5,38] is a two-party secure protocol that enables two parties to get the additive secret shares of a permuted vector. After execution of ST protocol, $\mathcal{S}$ holds two vectors $\{a_i\}_{i\in[n]}$, $\{\langle a_{\pi(i)} \rangle_1\}_{i\in[n]}$ while $\mathcal{R}$ holds a permutation π and the vector $\{\langle a_{\pi(i)} \rangle_0\}_{i\in[n]}$. We use ST to achieve secret sharing of the permuted items in 2PC. The formal functionality is given for reference in Appendix A.

Homomorphic Encryption. Homomorphic encryption (HE) [2, 16, 19, 29] enables arithmetic computations to be carried out directly on encrypted data without decrypting the ciphertexts. We employ two classes of HE primitives: additive

homomorphic encryption (AHE) [29] for secret masking, and leveled fully homomorphic encryption (FHE) [2,16,19], which supports both homomorphic addition and multiplication, for polynomial evaluation in the encrypted domain.

2.3 Security Model

We use the standard notion of security in the presence of semi-honest adversaries. Let Π be a two-party protocol for computing the functionality $f(X, Y)$, where X and Y are the inputs of $\mathcal{S}$ and $\mathcal{R}$, and output(X, Y) be the output of both parties. Define $\text{View}_{\mathcal{S}}^{\Pi}(X, Y)$ and $\text{View}_{\mathcal{R}}^{\Pi}(X, Y)$ be the views of $\mathcal{S}$ and $\mathcal{R}$.

Definition 1. *Two-party protocol* Π *securely realize function* f *in the presence of semi-honest adversaries if for every* PPT *adversary* $\mathcal{A}$ *there exists a* PPT *simulator* Sim_S *and* Sim_R *such that for all inputs* X *and* Y*,*

$$\{\text{View}_{\mathcal{S}}^{\Pi}(X, Y), \text{output}(X, Y)\} \cong_c \{Sim_S(X, f(X, Y)), f(X, Y)\}$$
$$\{\text{View}_{\mathcal{R}}^{\Pi}(X, Y), \text{output}(X, Y)\} \cong_c \{Sim_R(Y, f(X, Y)), f(X, Y)\}$$

where $\cong_c$ *represents computational indistinguishability.*

3 Lightweight pmqPMT and Labeled Variant

We first introduce the ideal functionalities $\mathcal{F}_{\text{pmqPMT}}$ and its labeled variant $\mathcal{F}_{\text{pmqPMTwL}}$ in Figure 2. Then, we show their constructions in Section 3.1 and analyze the complexity and security in Section 3.2.

Functionality $\mathcal{F}_{\text{pmqPMT}}/\mathcal{F}_{\text{pmqPMTwL}}$

Input: $\mathcal{F}_{\text{pmqPMT}}/\mathcal{F}_{\text{pmqPMTwL}}$ gets $X = \{x_i\}_{i\in[n_x]}$ ($X = \{x_i, u_i\}_{i\in[n_x]}$ for labeled case) from $\mathcal{S}$ and $Y = \{y_i\}_{i\in[n_y]}$ from $\mathcal{R}$. ($n_y \ll n_x$)

Functionality $\mathcal{F}_{\text{pmqPMT}}$:

1) $\mathcal{F}_{\text{pmqPMT}}$ samples a permutation π over $[n_y]$ and defines a vector $\boldsymbol{e} \in \{0,1\}^{n_y}$, where $\forall\ i \in [n_y]$, if $\exists\ j \in [n_x]$ s.t. $y_{\pi(i)} = x_j$, $e_i = 1$, else $e_i = 0$.
2) $\mathcal{F}_{\text{pmqPMT}}$ outputs π to the $\mathcal{S}$ and vector $\boldsymbol{e} = \{e_i\}_{i\in[n_y]}$ to $\mathcal{R}$.

Functionality $\mathcal{F}_{\text{pmqPMTwL}}$:

1) $\mathcal{F}_{\text{pmqPMT}}$ samples a permutation π over $[n_y]$ and defines the vectors $\boldsymbol{e} \xleftarrow{\$} \{0,1\}^{n_y}$, $\boldsymbol{r} \xleftarrow{\$} \mathbb{F}^{n_y}$. For each $i \in [n_y]$, if $\exists\ j \in [n_x]$ s.t. $y_{\pi(i)} = x_j$, then $e_i = 1$, $r_i = u_j$. Otherwise, $e_i = 0$ and r_i is a random value.
2) $\mathcal{F}_{\text{pmqPMT}}$ outputs π and $\langle \boldsymbol{r} \rangle_1$ to the $\mathcal{S}$, and $\boldsymbol{e} = \{e_i\}_{i\in[n_y]}$, $\langle \boldsymbol{r} \rangle_0$ to $\mathcal{R}$.

Fig. 2. Ideal functionalities $\mathcal{F}_{\text{pmqPMT}}/\mathcal{F}_{\text{pmqPMTwL}}$ of the pmqPMT/pmqPMTwL.

3.1 Detailed Constructions

Revisit of FHE-based PSI [6,10]. To illustrate our techniques, we first revisit the FHE-based unbalanced PSI protocol [10], which typically operates in two phases to handle large datasets efficiently:

1. **Item Mapping (OPRF Phase)**: The sender samples an OPRF key k and computes $X' = \{F_k(x) \mid x \in X\}$. Then, both parties invoke the DH-based OPRF and help the receiver obtain $Y' = \{F_k(y) \mid y \in Y\}$ without knowing k. Now computing $X \cap Y$ amounts to computing $X' \cap Y'$. This step allows the subsequent FHE operations to work with smaller, more efficient parameters.
2. **Membership Check (FHE-based Evaluation Phase)**: The Sender constructs the polynomial $f(x)$ such that for all $x' \in X'$, $f(x') = 0$. After receiving the ciphertext $[\![y']\!]$ from the receiver, the Sender homomorphically evaluates the polynomial $f([\![y']\!])$ and returns the result to the receiver, who decrypts it and checks whether $f(y') = 0$. If and only if $y' \in X'$, $f(y') = 0$.

Based on the SOTA uPSI, we present the construction details of pmqPMT (Π_{pmqPMT}) and its labeled variant (Π_{pmqPMTwL}) in Figure 3.

Protocol Π_{pmqPMT}. Our construction is driven by a fundamental observation: the result of FHE-based membership checks inherently indicates the status of a specific mapped item y'. To conclude whether the original item y belongs to the intersection, the receiver must maintain the correspondence between y' and y. Consequently, we observe that by applying a secret permutation to the link between the mapped set Y' and the original set Y, we can effectively prevent the receiver from tracing the results back to the original inputs. This simple yet effective mechanism hides the intersection positions, thereby transforming a standard PSI protocol into a pmqPMT.

Based on this insight, we modify the item mapping phase by replacing the standard OPRF with a permuted OPRF (pOPRF). Specifically, the sender samples a secret permutation π. During the OPRF interaction, the receiver obtains the mapped values in a shuffled order $Y' = \{F_k(y_{\pi(i)})\}_{i \in [n_y]}$ without knowing the permutation π. Subsequently, the standard FHE membership check is performed on Y'. The receiver obtains a bit vector $\mathbf{e}$, where $e_i = 1$ indicates a match. However, since the input order was shuffled by π (unknown to the receiver), the receiver cannot map any e_i back to a specific y_j. This successfully hides the intersection elements with negligible overhead, as the pOPRF merely adds a local permutation to the efficient DH-based OPRF.

Protocol Π_{pmqPMTwL}. We employ the interpolation polynomials to realize the pmqPMT with labels protocol Π_{pmqPMTwL}. $\mathcal{S}$ constructs a polynomial G such that for all $x_i \in X$, $G(x_i) = u_i$, and $G(x)$ is a random number for any $x \in \mathbb{F} \backslash X$. Given an encryption $[\![y'_i]\!]$ by FHE, $\mathcal{S}$ could evaluate $G([\![y'_i]\!])$ homomorphically, mask the result by a random value $\langle r_i \rangle_1 \xleftarrow{\$} \mathbb{F}$, and return $G([\![y'_i]\!]) - \langle r_i \rangle_1$. The $\mathcal{R}$ then decrypts $\langle r_i \rangle_0 = G(y'_i) - \langle r_i \rangle_1$, which is either subject to $r_i = u_j$ if $y'_i = x'_j$, or uniformly random in $\mathbb{F}$.

Other Optimizations. The efficiency of Π_{pmqPMT} and Π_{pmqPMTwL} can be significantly enhanced by integrating standard optimizations for FHE-based membership tests [6, 7, 10], such as hashing, batching, and the Paterson-Stockmeyer algorithm. Furthermore, our DH-based pOPRF design is particularly advantageous for cloud scenarios where the Sender holds a large, static dataset. By utilizing a static OPRF key, the Sender can perform the computationally in-

Protocol $\Pi_{\text{pmqPMT}}/\Pi_{\text{pmqPMTwL}}$

Input: The receiver ($\mathcal{R}$) has an input set $Y = \{y_i\}_{i\in[n_y]}$. The sender ($\mathcal{S}$) has an input set $X = \{x_i, u_i\}_{i\in[n_x]}$. ($n_y \ll n_x$).
Protocol:

I. **Offline Phase:**

1. [**Setup**]: $\mathcal{S}$ and $\mathcal{R}$ agree on FHE parameters and an OPRF function F.
2. [**OPRF**]: $\mathcal{S}$ samples a key k and precomputes the OPRF result of each item $x \in X$ as $x' = F_k(x)$.
3. [**Coefficient Computation**]: $\mathcal{S}$ constructs the polynomial $P(x) = a_0 \cdot x^{n_x} + \cdots + a_{n_x}$ such that $P(x'_i) = 0$ for all $x'_i \in X'$.
   (a). ***[Labeled Polynomial]** If $\mathcal{S}$ has labels associated with its set X, then it interpolates the polynomial $G(x) = b_0 \cdot x^{n_x} + \cdots + b_{n_x}$ subject to $G(x'_i) = u_i$ where u_i and x'_i are the labeled value and the OPRF result of the item $x_i \in X$.

II. **Online Phase:**

1. [**Permuted OPRF**] $\mathcal{R}$ obtains the $Y' = \{F_k(y_{\pi(i)})\}_{i\in[n_y]}$ through interacting with $\mathcal{S}$, who secretly holds the permutation π.
2. **[Receiver Query]** $\mathcal{R}$ uses the public key pk to encrypt each item $y'_i \in Y'$ and sends all ciphertexts $[\![y'_i]\!]$ ($i \in [n_y]$) to $\mathcal{S}$.
3. **[Polynomial Evaluation]** $\mathcal{S}$ receives the ciphertexts and computes all powers of the encryption $[\![y'_i]\!], ..., [\![(y'_i)^{n_x}]\!]$ for each $[\![y'_i]\!]$. Then, $\mathcal{S}$ samples non-zero random numbers $\{\alpha_i\}_{i\in[n_y]}$ and homomorphically evaluates $P([\![y'_i]\!])$ by $[\![z_i]\!] = \alpha_i \cdot \sum_{j=0}^{n_x} \left(a_j \cdot [\![(y'_i)^{(n_x-j)}]\!]\right)$
   (a). ***[Labeled Evaluation]** $\mathcal{S}$ samples random numbers $\{r'_i\}_{i\in[n_y]}$ and performs the same operation to homomorphically evaluate $G([\![y'_i]\!])$ by $[\![g_i]\!] = \sum_{j=0}^{n_x} (b_j \cdot [\![(y'_i)]\!]) - r'_i$.
   All evaluation results are sent back to $\mathcal{R}$.
5. **[Outputs]** $\mathcal{R}$ decrypts all ciphertexts $[\![z_i]\!]$ to z_i ($i \in [n_y]$). $\mathcal{R}$ defines an all-zero vector $\mathbf{e} = \{e_i\}_{i\in[n_y]}$. For each $i \in [n_y]$, if $z_i = 0$, $\mathcal{R}$ sets $e_i = 1$. Finally, $\mathcal{R}$ outputs $\mathbf{e} = \{e_i\}_{i\in[n_y]}$ and $\mathcal{S}$ outputs the π.
   (a). ***[Labeled Output]** $\mathcal{R}$ decrypts ciphertext $[\![g_i]\!]$ to $\langle r_i\rangle_0$, $i \in [n_y]$. Then, $\mathcal{R}$ and $\mathbf{S}$ output vectors $\langle \mathbf{r}\rangle_0 = \{\langle r_i\rangle_0\}_{i\in[n_y]}$ and $\langle \mathbf{r}\rangle_1 = \{r'_i\}_{i\in[n_y]}$, respectively.

Fig. 3. Constructions of protocols Π_{pmqPMT} and Π_{pmqPMTwL}. An asterisk (*) indicates steps are only required for Π_{pmqPMTwL}.

tensive preprocessing (i.e., OPRF evaluation and polynomial interpolation) only once. *This precomputed state can then be reused across multiple query requests from different Receivers, substantially reducing the amortized cost.*

3.2 Complexity & Security Analysis

Communication Complexity. Π_{pmqPMT} and Π_{pmqPMTwL} involve communication in the pOPRF and FHE-based polynomial evaluation. The number of messages transmitted in the pOPRF execution is linear with the size of Y,

denoted as $\mathcal{O}(n_y)$. Therefore, it has the communication complexity of $\mathcal{O}(n_y)$. Employing the optimizations of [6, 7, 10], the communication complexity of the FHE-based polynomial evaluation is $\mathcal{O}(n_y \log n_x)$. In summary, the communication complexity of Π_{pmqPMT} and Π_{pmqPMTwL} is $\mathcal{O}(n_y \log n_x)$.

Security. Note that $\Pi_{\text{pmqPMT}}/\Pi_{\text{pmqPMTwL}}$ do not protect the cardinality of the intersection from $\mathcal{R}$, which is inherent in ideal functionalities. Nevertheless, as discussed in [8, 18, 40], the fine-grained functions that need to be evaluated already contain the intersection cardinality. For instance, the secret-sharing function, which is proposed to support the generic 2PC computation, leaks the cardinality to both $\mathcal{R}$ and $\mathcal{S}$. Furthermore, the intersection cardinality is frequently the result that participants aim to get for certain applications [3, 23]. Lemma 1 captures the security of the Π_{pmqPMT} and Π_{pmqPMTwL}.

Lemma 1. *If the DH assumption holds and the FHE scheme is IND-CPA secure, the protocol* Π_{pmqPMT} *(resp.* Π_{pmqPMTwL}*) of Figure 3 securely realizes the* $\mathcal{F}_{\text{pmqPMT}}$ *(resp.* Π_{pmqPMTwL}*) functionality against semi-honest adversaries in the random oracle model.*

4 Unbalanced Private Computation on Set Intersection

In this section, we first present how to construct uPCSI from our proposed pmqPMT to support functions: i) card: outputs $k = |X \cap Y|$ to $\mathcal{R}$; ii) card-sum: outputs $(k = |X \cap Y|,\ s = \sum_{z_i \in X \cap Y} z_i)$ to $\mathcal{R}$; and iii) Return shares $\{\langle z_{\pi(i)} \rangle_0\}_{i \in [k]}$ to $\mathcal{R}$ and $\{\langle z_{\pi(i)} \rangle_1\}_{i \in [k]}$ to $\mathcal{S}$, where z_i is a random permutation of $X \cap Y$ and $k = |X \cap Y|$, and then analyze the complexity and security.

4.1 uPCSI Construction from pmqPMT

Figure 4 illustrates the formal uPCSI protocol design upon the pmqPMT, and we explain it as follows:

uPCSI-card. With the outputs $\{e_i\}_{i \in [n_y]}$ of Π_{pmqPMT}, $\mathcal{R}$ could trivially computes the intersection cardinality by $\sum_{i=0}^{n_y} e_i$.

uPCSI-card-sum. With the private input Y, the output $\{e_i\}_{i \in [n_y]}$ held by $\mathcal{R}$ and the permutation method π held by $\mathcal{S}$, the sum of the intersection can be computed as $sum = \sum_{i=0}^{n_y} e_i \cdot y_{\pi(i)}$. To achieve this computation, two parties first invoke $\mathcal{F}_{\text{ST}}$ to generate the ST tuples $a_{\pi(i)} = \langle a_{\pi(i)} \rangle_0 + \langle a_{\pi(i)} \rangle_1, i \in [n_y]$, where $\mathcal{S}$ holds input π, receives the output $\{\langle a_{\pi(i)} \rangle_1\}_{i \in [n_y]}$ and $\mathcal{R}$ receives the outputs $\{\langle a_{\pi(i)} \rangle_0\}_{i \in [n_y]}$, $\{a_i\}_{i \in [n_y]}$. Then, $\mathcal{R}$ sends masked values $\{y'_i = y_i - a_i\}_{i \in [n_y]}$ to $\mathcal{S}$, who permutes them according to π. This allows us to rewrite *sum* as:

$$sum = \sum_{i=0}^{n_y} e_i \cdot \left(y_{\pi(i)} - a_{\pi(i)}\right) = \sum_{i=0}^{n_y} e_i \cdot \langle a_{\pi(i)} \rangle_0 + \sum_{i=0}^{n_y} e_i \cdot \left(y'_{\pi(i)} + \langle a_{\pi(i)} \rangle_1\right)$$

where the first term $(\sum_{i=0}^{n_y} e_i \cdot \langle a_{\pi(i)} \rangle_0)$ can be locally computed by $\mathcal{R}$. Therefore, we only need to securely compute the $\sum_{i=0}^{n_y} e_i \cdot \left(y'_{\pi(i)} + \langle a_{\pi(i)} \rangle_1\right)$.

Protocol Π_{uPCSI}

Input: $\mathcal{S}$ inputs $X = \{x_i\}_{i\in[n_x]}$ and $\mathcal{R}$ inputs $Y = \{y_i\}_{i\in[n_y]}$, $(n_y \ll n_x)$.

Protocol:

1. $\mathcal{S}$ and $\mathcal{R}$ play as the sender and the receiver to invoke the protocol Π_{pmqPMT}: $\mathcal{S}$ and $\mathcal{R}$ input X and Y, and get a permutation π and $\{e_i\}_{i\in[n_y]}$, respectively.

I. **uPCSI-card**

2. $\mathcal{R}$ computed and output $k = \sum_{i=1}^{n_y} e_i$.

II. **uPCSI-card-sum**

2. $\mathcal{S}$ and $\mathcal{R}$ execute the protocol $\mathcal{F}_{\text{ST}}$, where $\mathcal{S}$ holds input π, receives the output $\{\langle a_{\pi(i)}\rangle_1\}_{i\in[n_y]}$ and $\mathcal{R}$ receives $\{\langle a_{\pi(i)}\rangle_0\}_{i\in[n_y]}$, $\{a_i\}_{i\in[n_y]}$.
3. $\mathcal{R}$ computes $y'_i = y_i - a_i, i \in [n_y]$ and sends them to $\mathcal{S}$.
4. $\mathcal{S}$ permutes $\{y'_i\}_{i\in[n_y]}$ with π and gets $\{y'_{\pi(i)}\}_{i\in[n_y]}$. Then, $\mathcal{S}$ samples the random vector $\{w_i\}_{i\in[n_y]}$ such that $\sum_{i=1}^{n_y} w_i = 0$ and constructs pairs $(m_i^0, m_i^1) = (w_i, y'_{\pi(i)} + w_i + \langle a_{\pi(i)}\rangle_1)$.
5. $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\text{OT}}$: For $i \in [n_y]$, $\mathcal{S}$ inputs (m_i^0, m_i^1) and $\mathcal{R}$ inputs choice bit e_i. As a result, $\mathcal{R}$ gets $m_i^{e_i}$ and outputs $sum = \sum_{i=1}^{n_y}(m_i + e_i \cdot \langle a_{\pi(i)}\rangle_0)$.

III. **uPCSI-secret-sharing**

2. $\mathcal{S}$ and $\mathcal{R}$ execute the protocol $\mathcal{F}_{\text{ST}}$ with length n_y twice. In the first execution, $\mathcal{S}$ inputs π, receives $\{\langle a_{\pi(i)}\rangle_1\}_{i\in[n_y]}$, and $\mathcal{R}$ receives $\{\langle a_{\pi(i)}\rangle_0\}_{i\in[n_y]}, \{a_i\}_{i\in[n_y]}$. In the second execution, $\mathcal{R}$ inputs a random permutation π^*, receives $\{\langle b_{\pi^*(i)}\rangle_0\}_{i\in[n_y]}$, and $\mathcal{S}$ receives output $\{\langle b_{\pi^*(i)}\rangle_1\}_{i\in[n_y]}$, $\{b_i\}_{i\in[n_y]}$.
3. $\mathcal{R}$ sets $y'_i = y_i - a_i$, $e_i^* = e_{\pi^*(i)}$, $i \in [n_y]$ and sends the results to $\mathcal{S}$.
4. $\mathcal{S}$ permutes $\{y'_i\}_{i\in[n_y]}$ with π, computes $y_i^* = y'_{\pi(i)} + \langle a_{\pi(i)}\rangle_1 - b_i, i \in [n_y]$, and sends $\{y_i^*\}_{i\in[n_y]}$ back to $\mathcal{R}$.
5. $\mathcal{R}$ permutes $\{y_i^*\}_{i\in[n_y]}$ with π^* and sets $y_i^0 = y^*_{\pi^*(i)} + \langle a_{\pi^*(\pi(i))}\rangle_0 + \langle b_{\pi^*(i)}\rangle_0$. For all $i \in [n_y]$, if $e_i^* = 0$, $\mathcal{R}$ removes y_i^0. Otherwise, it defines $\langle z_j\rangle_0 = y_i^0$, $j \in [k]$, *s.t.* $k = \sum_{i=1}^{n_y} e_i^*$. Finally, $\mathcal{R}$ outputs $\{\langle z_j\rangle_0\}_{j\in[k]}$.
6. $\mathcal{S}$ sets $y_i^1 = \langle b_{\pi^*(i)}\rangle_1$, $i \in [n_y]$. If $e_i^* = 0$, $\mathcal{S}$ removes y_i^1. Otherwise, it defines $\langle z_j\rangle_1 = y_i^1$, $j \in [k]$, *s.t.* $k = \sum_{i=1}^{n_y} e_i^*$. Then, $\mathcal{S}$ outputs $\{\langle z_j\rangle_1\}_{j\in[k]}$.

Fig. 4. Full protocol of uPCSI framework.

To achieve this, we use the OT protocol in the second step. $\mathcal{S}$ first samples a random vector $\boldsymbol{w} = \{w_i\}_{i\in[n_y]}$, s.t. $\sum_{i=0}^{n_y} w_i = 0$. Then, they invoke n_y instances of OT, where $\mathcal{S}$ inputs $(m_i^0, m_i^1) = (y'_{\pi(i)} + \langle a_{\pi(i)}\rangle_1 + w_i, w_i)$, and $\mathcal{R}$ inputs e_i as the choice bit. As a result, $\mathcal{R}$ receives $m_i^{e_i} = e_i \cdot (y'_{\pi(i)} + \langle a_{\pi(i)}\rangle_1) + w_i$. Finally, $\mathcal{R}$ outputs $\sum_{i=0}^{n_y} m_i^{e_i} + \sum_{i=0}^{n_y} e_i \cdot \langle a_{\pi(i)}\rangle_0$, which equals the desired *sum*.

uPCSI-secret-sharing. To realize oblivious alignment without leaking the secret permutation π, we introduce a dual-shuffling mechanism using two $\mathcal{F}_{\text{ST}}$ instances. The first ST execution generates secret shares of Y under the pOPRF permutation π, held by $\mathcal{S}$. Given the ST tuples $a_{\pi(i)} = \langle a_{\pi(i)}\rangle_0 + \langle a_{\pi(i)}\rangle_1, i \in [n_y]$,

where $\mathcal{S}$ holds $\{\langle a_{\pi(i)}\rangle_1\}_{i\in[n_y]}$ and $\mathcal{R}$ holds $\{\langle a_{\pi(i)}\rangle_0\}_{i\in[n_y]}$, $\{a_i\}_{i\in[n_y]}$. Then, $\mathcal{R}$ computes and sends $y'_i = y_i - a_i, i \in [n_y]$ to $\mathcal{S}$, who permutes the $\{y'_i\}_{i\in[n_y]}$ and computes $y'_{\pi(i)} + \langle a_{\pi(i)}\rangle_1$. Now $\{\langle a_{\pi(i)}\rangle_0\}_{i\in[n_y]}$ and $\{y'_{\pi(i)} + \langle a_{\pi(i)}\rangle_1\}_{i\in[n_y]}$ are the shares of $\{y_{\pi(i)}\}_{i\in[n_y]}$. To enable $\mathcal{S}$ filtering the intersection items obliviously, a second ST execution is invoked where $\mathcal{R}$ inputs a fresh random permutation π^*. This secondary shuffle lets two parties get the shares of $\{y_{\pi^*(\pi(i))}\}_{i\in[n_y]}$ and transforms the indicator vector e into $e^* = \{e_{\pi^*(i)}\}$, which is then revealed to $\mathcal{S}$. Consequently, $\mathcal{S}$ can locally select the intersection shares corresponding to $e^*_i = 1$ without tracing them back to original indices.

4.2 Complexity & Security Analysis

Communication Complexity: We analyze the communication complexity of each protocol within the uPCSI framework, excluding the Π_{pmqPMT} protocol, whose complexity has been shown in section 3.1. First, the uPCSI-card requires no communication except for the pmqPMT protocol. Then, since the protocol $\mathcal{F}_{\text{ST}}$ with length n_y takes $\mathcal{O}(n_y \log n_y)$ communication complexity and both the n_y instances 1-out-of-2 OT and sending messages $\{y'_i\}_{i\in[n_y]}$ take $\mathcal{O}(n_y)$ complexity, the overall communication complexity of the uPCSI-card-sum protocol-excluding Π_{pmqPMT}-is $\mathcal{O}(n_y \log n_y)$. Similarly, the main communication cost of uPCSI-secret-sharing stems from the execution of $\mathcal{F}_{\text{ST}}$ and sending the vectors $\{y'_i\}_{i\in[n_y]}, \{y^*_i\}_{i\in[n_y]}$. Hence, excluding Π_{pmqPMT}, the communication complexity of uPCSI-secret-sharing also is $\mathcal{O}(n_y \log n_y)$.

Security. We capture the security of the uPCSI framework in Lemma 2.

Lemma 2. *The uPCSI framework in Figure 4 (including the protocols of uPCSI-card, uPCSI-card-sum, and uPCSI-secret-sharing) securely computes the corresponding functions, i.e., intersection cardinality, intersection sum with cardinality, and secret-sharing of the intersection, against semi-honest adversaries in the ($\mathcal{F}_{\text{pmqPMT}}, \mathcal{F}_{\text{ST}}, \mathcal{F}_{\text{OT}}$)-hybrid model.*

5 Labeled uPCSI

In this section, we extend the uPCSI to the labeled case, including the single-labeled (SL)-uPCSI (section 5.1) and double-labeled (DL)-uPCSI (section 5.2).

5.1 Single-Labeled uPCSI

The cardinality of the intersection is the same as the basic uPCSI, as shown in section 4, we mainly construct SL-uPCSI from pmqPMTwL to support two functions: i) card-sum: outputs $k = |X \cap Y|$, $s = \sum_{i=0}^{k}(u_i)$, where u_i is the label values associated to the intersection item, to $\mathcal{R}$; and ii) return shares $\{\langle z_{\pi(i)}\rangle_0\}_{i\in[k]}$ to $\mathcal{R}$ and $\{\langle z_{\pi(i)}\rangle_1\}_{i\in[k]}$ to $\mathcal{S}$, where z_i is a random permutation of $\{u_i\}_{i\in[k]}$ and $k = |X \cap Y|$, and then analyze complexity and security.

Protocol $\Pi_{\text{SL-uPCSI}}$

Input: $\mathcal{R}$ inputs $Y = \{y_i\}_{i\in[n_y]}$. $\mathcal{S}$ inputs $X = \{x_i, u_i\}_{i\in[n_x]}$. $(n_y \ll n_x)$

Protocol:

1. $\mathcal{S}$ and $\mathcal{R}$ play as the sender and the receiver to invoke Π_{pmqPMTwL} with input X and Y. As a result, $\mathcal{S}$ gets a permutation π over $[n_y]$ and a vector $\{r_i^1\}_{i\in[n_y]}$, $\mathcal{R}$ gets vectors $\{e_i\}_{i\in[n_y]}$ and $\{r_i^0\}_{i\in[n_y]}$.

I. **card-sum**

2. $\mathcal{S}$ defines pairs $(m_i^0, m_i^1) = (w_i, w_i + r_i^1)$, where $i \in [n_y]$, $\sum_{i=1}^{n_y} w_i = 0$.
3. $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\text{OT}}$: $\mathcal{S}$ inputs (m_i^0, m_i^1) and $\mathcal{R}$ inputs choice bit e_i. The result is that $\mathcal{R}$ gets $\{m_i^{e_i}\}_{i\in[n_y]}$ and outputs $sum = \sum_{i=1}^{n_y} \left(e_i \cdot r_i^0 + m_i^{e_i}\right)$.

II. **secret-sharing**

2. $\mathcal{S}$ and $\mathcal{R}$ invoke the protocol $\mathcal{F}_{\text{ST}}$. $\mathcal{R}$ inputs a random permutation π^*, receives $\{\langle a_{\pi^*(i)}\rangle_0\}_{i\in[n_y]}$, and $\mathcal{S}$ receives output $\{\langle a_{\pi^*(i)}\rangle_1\}_{i\in[n_y]}, \{a_i\}_{i\in[n_y]}$.
3. $\mathcal{S}$ computes $r_i' = r_i^1 - a_i, i \in [n_y]$, and sends $\{r_i'\}_{i\in[n_y]}$ back to $\mathcal{R}$.
4. $\mathcal{R}$ computes $r_i^* = r_i' + r_i^0, i \in [n_y]$ and permutes $\{r_i^*\}_{i\in[n_y]}$, $\{e_i\}_{i\in[n_y]}$ with π^*. For $i \in [n_y]$, if $e_{\pi^*(i)} = 0$, $\mathcal{R}$ removes $r_{\pi^*(i)}^*$. Otherwise, it sets $\langle z_j\rangle_0 = r_{\pi^*(i)}^* + \langle a_{\pi^*(i)}\rangle_0$, $j \in [k]$, where $k = \sum_{i=1}^{n_y} e_i$. Finally, $\mathcal{R}$ sends $\{e_{\pi^*(i)}\}_{i\in[n_y]}$ to $\mathcal{S}$ and outputs $\{\langle z_j\rangle_0\}_{j\in[k]}$, permutation π^*.
5. For $i \in [n_y]$, if $e_{\pi^*(i)} = 0$, $\mathcal{S}$ removes $\langle a_{\pi^*(i)}\rangle_1$. Otherwise, it defines $\langle z_j\rangle_1 = \langle a_{\pi^*(i)}\rangle_1, j \in [k]$ ($k = \sum_{i=1}^{n_y} e_i^*$). Finally, $\mathcal{S}$ outputs $\{\langle z_j\rangle_1\}_{j\in[k]}$.

Fig. 5. Full protocol of SL-uPCSI with $\mathcal{S}'$ labels.

SL-uPCSI with $\mathcal{R}$'s labels. The SL-uPCSI with $\mathcal{R}$'s labels could be directly realized by uPCSI in section 4 as long as $\mathcal{R}$ uses its label values u_i to replace y_i after the execution of Π_{pmqPMT}.

SL-uPCSI with $\mathcal{S}$'s labels. We now construct the SL-uPCSI with $\mathcal{S}$'s labels from protocol Π_{pmqPMTwL}, who output $\mathbf{e}, \mathbf{r}^0$ and $\pi, \mathbf{r}^1$ to $\mathcal{R}$ and $\mathcal{S}$, respectively.

- In the basic uPCSI-card-sum, the parties first use ST tuples to permute-and-share $\{y_i\}_{i\in[n_y]}$, and then apply OT protocol to compute the *sum*. In SL-uPCSI-card-sum, the labels are already secret-shared after Π_{pmqPMTwL}, enabling $\mathcal{R}$ to compute the *sum* via OT in the same way.
- The basic uPCSI-secret-sharing uses two ST tuples: one for secretly sharing items $\{y_i\}_{i\in[n_y]}$ and another for selecting shares of the intersection. In SL-uPCSI-secret-sharing, since the labels are already secret-shared, the parties directly use one ST tuple to select the shares, following the same method as in section 4.

The full protocol is shown in Figure 5. It is trivial to see that the card-sum (resp. secret-sharing) function of SL-uPCSI with $\mathcal{S}$'s labels takes $\mathcal{O}(n_y)$ (resp. $\mathcal{O}(n_y \log n_y)$) communication complexity. Lemma 3 captures the security of SL-uPCSI with $\mathcal{S}$'s labels. The proof of Lemma 3 is similar to uPCSI of section 4.1.

Protocol $\Pi_{\text{DL-uPCSI}}$

Input: $\mathcal{R}$ inputs $Y = \{y_i, v_i\}_{i\in[n_y]}$. $\mathcal{S}$ inputs $X = \{x_i, u_i\}_{i\in[n_x]}$. $(n_y \ll n_x)$

Protocol:

1. $\mathcal{S}$ and $\mathcal{R}$ play as the sender and the receiver to invoke the protocol Π_{pmqPMTwL} with input X and Y.As a result, $\mathcal{S}$ gets a permutation π and $\mathbf{r}^1 = \{r_i^1\}_{i\in[n_y]}$. $\mathcal{R}$ gets $\mathbf{e} = \{e_i\}_{i\in[n_y]}$ and $\mathbf{r}^0 = \{r_i^1\}_{i\in[n_y]}$.
2. Then, $\mathcal{R}$ and $\mathcal{S}$ execute the SL-uPCSI-secret-share with $\mathcal{R}'$ labels to share the labels $\{v_i\}_{i\in[n_y]}$. As a result, $\mathcal{R}$ gets $\{\langle v_j\rangle_0\}_{j\in[k]}$ and permutation π^*, while $\mathcal{S}$ gets $\{\langle v_j\rangle_1\}_{j\in[k]}$. $(k = |X \cap Y|)$.
3. $\mathcal{S}$ encrypts $\{r_i^1\}_{i\in[n_y]}$ as $[\![c_i^r]\!] = \text{AHE.Enc}(pk_{\mathcal{S}}, r_i^1)$ and sends results to $\mathcal{R}$.
4. $\mathcal{R}$ permutes $\{e_i\}_{i\in[n_y]}$, $\{r_i^1\}_{i\in[n_y]}$, $\{[\![c_i^r]\!]\}_{i\in[n_y]}$ by π^*. Then, for all $i \in [n_y]$, if $e_{\pi^*(i)} = 1$, $\mathcal{R}$ defines $[\![c_j']\!] = [\![c_{\pi^*(i)}^r]\!] + r_{\pi^*(i)}^0 - \delta_j$, $\langle u_j\rangle_0 = \delta_j$, inwhere $\{\delta_j\}_{j\in[k]}$ are random numbers. Then, $\mathcal{R}$ returns ciphertexts $[\![c_j']\!]_{j\in[k]}$ to $\mathcal{S}$ and outputs shares $\{\langle u_j\rangle_0\}_{j\in[k]}$ and $\{\langle v_j\rangle_0\}_{j\in[k]}$.
5. $\mathcal{S}$ decrypts $[\![c_j']\!]$ to $\langle u_j\rangle_1$, $j \in [k]$, and outputs $\{\langle u_j\rangle_1\}_{j\in[k]}$, $\{\langle v_j\rangle_1\}_{j\in[k]}$.

Fig. 6. Full protocol of DL-uPCSI.

Lemma 3. *The SL-uPCSI with $\mathcal{S}$'s labels in Figure 5 are secure against semi-honest adversaries in $(\mathcal{F}_{\text{pmqPMTwL}}, \mathcal{F}_{\text{OT}}, \mathcal{F}_{\text{ST}})$-hybrid model.*

5.2 Double-Labeled uPCSI

In this section, we present DL-uPCSI, where $\mathcal{S}$ and $\mathcal{R}$ input $\{x_i, u_i\}_{i\in[n_x]}$ and $\{y_i, v_i\}_{i\in[n_y]}$, respectively, and want to compute functions on the labeled values (u_i, v_i) of the intersection items. Since the uPCSI-card-sum protocol only computes the sum of the label values from either $\mathcal{S}'$ or $\mathcal{R}'$, which can be realized by SL-uPCSI-card-sum directly, we focus primarily on the secret-sharing function [4] in the context of DL-uPCSI.

DL-uPCSI-secret-sharing. To support double-labeled inputs where both parties hold sensitive attributes $\{u_i\}$ and $\{v_i\}$, we extend the secret-sharing framework via a nested permutation-alignment mechanism. We first invoke the uPCSI-secret-sharing protocol to generate secret shares of the receiver's labels $\{v_j\}$ under the composite permutation $\pi^*(\pi)$. As for the sender's labels $\{u_i\}$, after the protocol Π_{pmqPMTwL}, they have been secretly shared between two parties as $\{r_i^0\}_{i\in[n_y]}$ and $\{r_i^1\}_{i\in[n_y]}$ in order of π. To align the sender's labels $\{u_i\}$ accordingly, the sender encrypts its shares using AHE. The receiver then applies the same secondary permutation π^* to these ciphertexts and reshares them. This approach ensures that shares of both u_j and v_j are synchronized in the same shuffled order $(\pi^*(\pi))$, effectively resolving the data alignment challenge in the double-labeled setting while maintaining sublinear communication complexity. The detailed protocol is illustrated in Figure 6.

Communication Complexity: Except for protocol Π_{pmqPMTwL}, DL-uPCSI-secret-sharing needs to perform the SL-uPCSI-secret-sharing with the label of

$\mathcal{R}$, which holds $\mathcal{O}(n_y \log n_y)$ complexity, and transfer the $\mathcal{O}(n_y)$ AHE ciphertexts. Therefore, except for the Π_{pmqPMTwL}, DL-uPCSI-secret-sharing takes $\mathcal{O}(n_y \log n_y)$ communication complexity.

Security. The security of DL-uPCSI protocols is captured in Lemma 4.

Lemma 4. *If the AHE scheme is IND-CPA secure, the protocol* $\Pi_{\text{DL-uPCSI}}$ *of Figure 6 correctly and securely realize the secret sharing function of DL-uPCSI against semi-honest adversaries in* $(\mathcal{F}_{\text{pmqPMTwL}}, \mathcal{F}_{\text{ST}})$*-hybrid model.*

6 Experiment

We implement our uPCSI framework in C++, and our implementation is available on `https://anonymous.4open.science/r/uPCSI-6E51/`. We benchmark it on an Intel(R) Xeon(R) Silver 4314 CPU with 2.40GHz and a 64GB RAM machine. For the FHE and AHE schemes, we use the BFV scheme [16] of the SEAL [36] and the Paillier scheme [29] of the Intel Paillier Cryptosystem Library (IPCL) [11]. The parameters for BFV scheme and corresponding security levels k follow those in [10, 40]. We implement the IKNP OT extension [24] using libOTe [30], and re-implement the SOTA ST protocol [5] on top of the libOTe. The bandwidth is set to be 10 Gbps with 20 ms round-trip time (RTT) latency for the LAN and 50 Mbps with 80 ms RTT latency for the WAN following [8,40], using the Linux tc command. All experiments set the input bit length $\ell = 128$ and are executed in a single thread by default. The statistical and computational security parameters are $\lambda = 40$ and $\kappa = 128$ bits.

Following [6, 10, 40, 41], the sender's sets are of size $\{2^{16}, 2^{20}, 2^{24}, 2^{28}\}$, while the receiver's set sizes range between 1024 and 11041. Note that with the hashing optimization of [7], approximately 1.33× more items of the receiver could be added with no difference in efficiency. However, following [7, 40], we adopt $n_y = 5535$ (resp. $n_y = 11041$) for the hash table size of 8192 (resp. 16384) to give a fair comparison with other protocols [8, 40] in terms of the runtime and communication cost. Since the overhead for pmqPMT is almost equal to that of uPCSI-card, we omit it.

6.1 Benchmarking uPCSI-card & card-sum

In this section, we compare our framework with the SOTA PCSI [8] and SOTA uPCSI [40] in terms of runtime and communication[5]. Following the experimental evaluations of [8], we focus on the performance of functions uPCSI-card and uPCSI-card-sum here, and present results for $n_y = 1024$ and $n_x \in \{2^{16}, 2^{20}\}$. Below, we evaluate our protocols in two settings: i) Classical 2PC setting. A receiver with a small set interacts with a sender possessing a large set to measure efficiency; ii) Multi-receiver setting. Multiple receivers (each with a small set) jointly execute the protocol with a single sender (holding a large set), demonstrating scalability and practical applicability.

[5] As no public implementations are available, we use the reported results in [40].

Table 1. Comparison with the SOTA uPCSI [40] and PCSI [8] protocols. Comm. (MB) includes the receiver-to-sender and sender-to-receiver communication costs. Our offline phase can be reused in multiple executions.

n_y	n_x	Protocol	Offline Time (s)	**uPCSI-card**			**uPCSI-card-sum**		
				Online Time (s)		Comm. (MB)	Online Time (s)		Comm. (MB)
				LAN	WAN		LAN	WAN	
1024	2^{16}	[8]	**0**	3.26	5.23	4.23	3.50	6.23	5.76
		[40]	**0**	1.65	5.71	2.08	1.75	6.25	2.18
		Ours	0.8	**0.48**	**1.60**	**1.34**	**0.68**	**2.09**	**1.69**
	2^{20}	[8]	**0**	52.58	66.10	67.70	54.95	73.69	91.70
		[40]	**0**	18.83	22.89	2.67	18.90	23.22	2.77
		Ours	27	**2.85**	**4.44**	**2.39**	**3.06**	**4.68**	**2.75**

2PC setting. The benchmarks in the classical 2PC setting are reported in Table 1. For the uPCSI-card protocol, our construction reduces the communication cost by 1.1-1.6× and 3.1-33.3× over [40] and [8], respectively. In terms of runtime, our protocol achieves 3.4-8.9× (resp. 3.4-6.1×) and 6.7-25.1× (resp. 3.1-15.9×) speedups over [40] and [8] in LAN (resp. WAN). This improvement arises from our efficient pmqPMT protocol. For the uPCSI-card-sum protocol, we achieve respective 2.6-8.1× (resp. 2.9-5.9×) and 5.1-23.6× (resp. 2.9-18.7×) runtime improvements compared with [40] and [8] in LAN (resp. WAN). This improvement stems from the efficiency benefits of the pmqPMT protocol and our lightweight uPCSI protocol.

Multi-Receiver Setting. We benchmark the protocols with $n_x = 2^{16}$ to demonstrate our efficiency in the multi-receiver setting. Since the offline phase of [40] cannot be reused across different receivers, we get their overhead in the multi-receiver setting by counting the costs of a single execution multiple times. Due to the page limit, we only compare the uPCSI-card protocol. For uPCSI-card-sum, one can refer to figure 10 in Appendix B. As shown in Figure 7, our protocol outperforms the prior solutions in runtime and communication cost: i) When the number of the receiver is 32, our uPCSI-card protocol saves the 35.5% communication cost, and is approximately 3.3× and 3.4× faster than that of [40] in LAN and WAN, respectively. ii) More importantly, the improvement shows a marked rise when the number of the receiver increases. These enhancements are due to the reusable offline phase and the lightweight online phase of our protocol. Besides, we improve the communication by 3.1× on average, and are upto 6.4× (5.6× on average) and 3.1× (2.9× on average) faster than the protocols of [8] in LAN and WAN. iii) In the unbalanced setting (a.k.a., $n_y \ll n_x$), our uPCSI framework has lower communication complexity $\mathcal{O}(n_y \log n_x)$ compared to [8] ($\mathcal{O}(n_x)$). That is why our communication overhead is reduced significantly.

6.2 Benchmarking in Billion-Scale

As highlighted in works [10, 20, 41], in practical applications such as advertising conversion rate evaluation, it is common for set sizes to reach millions or even billions. In this section, we report the performance of our protocols,

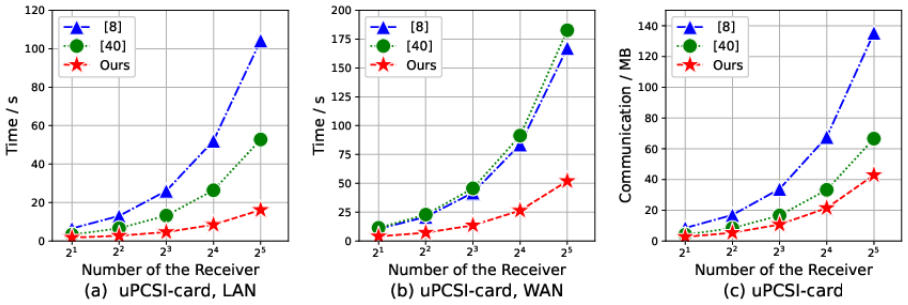

Fig. 7. Total cost (including online and offline phase) comparison among our uPCSI and SOTA works [8, 40] with $n_x = 2^{16}$ and $n_y = 1024$ in the multi-receiver setting.

including uPCSI-card, uPCSI-card-sum, and uPCSI-secret-share, with $n_x = \{2^{20}, 2^{24}, 2^{28}\}$. To our best knowledge, we are the first to perform uPCSI evaluation in this large setting.

As Table 2 shows, our uPCSI framework is efficient in computing a well-defined function on set intersection for large-scale datasets. For example, our uPCSI-card protocol only takes 25.32 seconds and 14.58 MB to compute the cardinality of the intersection between 2^{24} and 11041 items in LAN. Besides, the uPCSI-secret-sharing and uPCSI-card-sum protocols exhibit nearly identical runtime performance to the uPCSI-card protocols. Concretely, when $n_x = 2^{24}$, the average runtime of our uPCSI-secret-sharing and uPCSI-card-sum protocols is just 1.2 and 0.6 seconds longer than that of uPCSI-card. This feature mainly stems from the fact that we use the ST and OT, instead of AHE, to construct our uPCSI upon the pmqPMT protocol.

Our framework demonstrates superior scalability in massive-scale scenarios. For instance, computing the intersection between 2^{28} items and 2048 items requires only 16.56 MB of communication—a modest 2.5× increase compared to $n_x = 2^{20}$. The online phase completes within 10 seconds under a 32-thread configuration, underscoring its practical viability. Although our protocol incurs significant offline overhead, the reusable offline phase allows it to be executed only once for multiple protocol executions, making the amortized cost practical in real-world deployments. Notably, minor performance fluctuations observed across varying n_y are attributed to hashing bucket depths influencing the multiplicative depth of FHE evaluations, a phenomenon consistent with prior observations in [10].

7 Related works

We review existing works in terms of PCSI, including balanced and unbalanced settings, as follows.

Balanced Setting. Early arbitrary function evaluation over set intersections (circuit-PSI) relied on generic secure computation via circuit representation. Huang et al. [21] proposed an optimized sort-compare-shuffle circuit (via GMW

Table 2. Benchmarks of our uPCSI framework in the large-scale set size. When $n_x = 2^{28}$, all protocols are tested with 32 threads. Comm. (MB) includes the receiver-to-sender and sender-to-receiver communication costs. The offline phase can be reused.

n_x	n_y	Offline Time (s)	**uPCSI-card**			**uPCSI-card-sum**			**uPCSI-secret-share**		
			Online Time (s)		Comm. (MB)	Online Time (s)		Comm. (MB)	Online Time (s)		Comm. (MB)
			LAN	WAN		LAN	WAN		LAN	WAN	
2^{20}	2048	30	3.62	5.36	4.14	3.82	6.11	4.90	3.84	6.36	5.60
	4096	29	3.34	5.16	5.50	3.54	6.26	7.14	4.07	6.92	8.65
	5535	27	3.20	4.91	5.40	3.40	6.17	8.84	3.96	7.46	12.11
	11041	28	4.20	6.94	8.95	4.45	8.82	16.33	4.87	9.85	23.36
2^{24}	2048	871	21.34	23.95	5.18	21.52	24.20	5.94	21.76	24.69	6.64
	4096	864	21.80	24.36	7.60	22.07	26.24	9.24	22.70	26.96	10.75
	5535	871	23.10	26.93	8.50	23.76	27.30	11.94	24.03	28.29	15.21
	11041	832	27.32	32.20	14.58	28.02	33.10	21.95	28.45	34.64	28.99
2^{28}	1024	2487	8.21	11.13	12.57	9.42	12.51	12.93	9.57	12.96	13.25
	2048	2211	8.42	12.45	15.10	9.67	13.34	15.86	9.83	13.98	16.56

[27] or Yao's protocol [43]) with $\mathcal{O}(n \log n)$ complexity (n is the set size). Subsequent works [4, 9, 25, 31, 32] improved the computation and communication, and the works of [4, 25] achieved linear complexity. Recent works [34, 35] further boosted efficiency via oblivious key-value stores, setting the current SOTA. The PCSI framework [18] derives from the reverse private membership test (RPMT) [26], differing from circuit-PSI by supporting specific functions (PSI-card, PSI-card-sum). Garimella et al. [18] proposed a permuted characteristic variant of mqRPMT using oblivious switching networks (OSN) [28] for efficient PCSI, but OSN reliance leads to super-linear communication. The current SOTA PCSI [8] introduces multi-query RPMT (mqRPMT) based on decisional Diffie-Hellman (DDH)-like assumptions and achieves linear communication and computational complexity relative to the set size.

Unbalanced Setting. PCSI protocols are predominantly designed for balanced settings, where the two sets are of approximately equal size. However, in the unbalanced setting, where one set is significantly smaller than the other, the communication overhead becomes considerable compared to the non-private solution, which has communication linear in the size of the smaller set. The uPCSI has been proposed to alleviate this problem. Chen *et al.* [6] introduced a labeled PSI protocol with sublinear communication complexity in the size of the larger set, leveraging FHE. They further extended the protocol to enable both parties to obtain secret shares of labels associated with intersection items and demonstrated how to perform general 2PC computations on these shares. However, their work provides only theoretical descriptions without practical implementation. Besides, work [8] also presents the mqRPMT from the FHE-based PSI [6, 7, 10] for the unbalanced setting and gives the uPCSI framework. However, their protocols are incompatible with optimizations of [6,7,10], causing low efficiency. More recently, Tu *et al.* [40] formalized an ideal functionality named shared characteristic as an extension of previous (labeled) PSI works [6, 10]. By incorporating pmPEQT [39], they constructed the SOTA unbalanced pmqPMT

and subsequently presented the uPCSI framework. Their work also extended the uPCSI framework to support the single-labeled setting.

8 Conclusion

In this paper, we presented an efficient uPCSI framework built upon FHE-based PSI. Specifically, we designed a lightweight pmqPMT protocol and its labeled variant by leveraging the pOPRF within FHE-based PSI. Building on this construction, we developed efficient uPCSI protocols for various functionalities, including uPCSI-card, uPCSI-card-sum, and uPCSI-secret-sharing, while also supporting the double-labeled setting. Compared with existing approaches, our framework achieves superior scalability and efficiency. As future work, we plan to further enhance our protocols to attain malicious security.

References

1. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: CRYPTO. Springer (1992)
2. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. ACM TOCT (2014)
3. Buddhavarapu, P., Knox, A., Mohassel, P., Sengupta, S., Taubeneck, E., Vlaskin, V.: Private matching for compute. Cryptology ePrint Archive 2020/599 (2020)
4. Chandran, N., Gupta, D., Shah, A.: Circuit-psi with linear complexity via relaxed batch opprf. Proceedings on Privacy Enhancing Technologies (2022)
5. Chase, M., Ghosh, E., Poburinnaya, O.: Secret-shared shuffle. In: ASIACRYPT. Springer (2020)
6. Chen, H., Huang, Z., Laine, K., Rindal, P.: Labeled psi from fully homomorphic encryption with malicious security. In: ACM CCS (2018)
7. Chen, H., Laine, K., Rindal, P.: Fast private set intersection from homomorphic encryption. In: ACM CCS (2017)
8. Chen, Y., Zhang, M., Zhang, C., Dong, M., Liu, W.: Private set operations from multi-query reverse private membership test. In: IACR PKC. Springer (2024)
9. Ciampi, M., Orlandi, C.: Combining private set-intersection with secure two-party computation. In: SCN. pp. 464–482. Springer (2018)
10. Cong, K., Moreno, R.C., da Gama, M.B., Dai, W., Iliashenko, I., Laine, K., Rosenberg, M.: Labeled psi from homomorphic encryption with reduced computation and communication. ACM CCS (2021)
11. Corp., I.: Intel paillier cryptosystem library. `https://github.com/intel/palliercryptolib`
12. De Cristofaro, E., Gasti, P., Tsudik, G.: Fast and private computation of cardinality of set intersection and union. In: CANS. Springer (2012)
13. Demmler, D., Schneider, T., Zohner, M.: Aby-a framework for efficient mixed-protocol secure two-party computation. In: NDSS (2015)
14. Dong, C., Chen, L., Wen, Z.: When private set intersection meets big data: an efficient and scalable protocol. In: ACM CCS. pp. 789–800 (2013)
15. Duong, T., Phan, D.H., Trieu, N.: Catalic: Delegated psi cardinality with applications to contact tracing. In: ASIACRYPT. pp. 870–899. Springer (2020)

16. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive (2012)
17. Freedman, M.J., Ishai, Y., Pinkas, B., Reingold, O.: Keyword search and oblivious pseudorandom functions. In: TCC. pp. 303–324. Springer (2005)
18. Garimella, G., Mohassel, P., Rosulek, M., Sadeghian, S., Singh, J.: Private set operations from oblivious switching. In: IACR PKC. Springer (2021)
19. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: ACM STOC (2009)
20. Hetz, L., Schneider, T., Weinert, C.: Scaling mobile private contact discovery to billions of users. Cryptology ePrint Archive (2023)
21. Huang, Y., Evans, D., Katz, J.: Private set intersection: Are garbled circuits better than custom protocols? In: NDSS (2012)
22. Huberman, B.A., Franklin, M., Hogg, T.: Enhancing privacy and trust in electronic communities. In: 1st ACM conference on Electronic commerce (1999)
23. Ion, M., Kreuter, B., Nergiz, A.E., Patel, S., Saxena, S., Seth, K., Raykova, M., Shanahan, D., Yung, M.: On deploying secure computing: Private intersection-sum-with-cardinality. In: EuroS&P. IEEE (2020)
24. Ishai, Y., Kilian, J., Nissim, K., Petrank, E.: Extending oblivious transfers efficiently. In: CRYPTO. Springer (2003)
25. Karakoç, F., Küpçü, A.: Linear complexity private set intersection for secure two-party protocols. In: CANS. Springer (2020)
26. Kolesnikov, V., Rosulek, M., Trieu, N., Wang, X.: Scalable private set union from symmetric-key techniques. In: ASIACRYPT. Springer (2019)
27. Micali, S., Goldreich, O., Wigderson, A.: How to play any mental game. In: ACM STOC (1987)
28. Mohassel, P., Sadeghian, S.: How to hide circuits in mpc an efficient framework for private function evaluation. In: EUROCRYPT. pp. 557–574. Springer (2013)
29. Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: EUROCRYPT. pp. 223–238. Springer (1999)
30. Peter Rindal, L.R.: libOTe: an efficient, portable, and easy to use Oblivious Transfer Library. `https://github.com/osu-crypto/libOTe`
31. Pinkas, B., Schneider, T., Tkachenko, O., Yanai, A.: Efficient circuit-based psi with linear communication. In: EUROCRYPT. Springer (2019)
32. Pinkas, B., Schneider, T., Weinert, C., Wieder, U.: Efficient circuit-based psi via cuckoo hashing. In: EUROCRYPT. Springer (2018)
33. Rabin, M.O.: How to exchange secrets with oblivious transfer. Cryptology ePrint Archive (2005)
34. Raghuraman, S., Rindal, P.: Blazing fast psi from improved okvs and subfield vole. In: ACM CCS (2022)
35. Rindal, P., Schoppmann, P.: Vole-psi: fast oprf and circuit-psi from vector-ole. In: EUROCRYPT. pp. 901–930. Springer (2021)
36. Microsoft SEAL (release 4.0). `https://github.com/Microsoft/SEAL` (Mar 2022), microsoft Research, Redmond, WA.
37. Son, Y., Jeong, J.: Psi with computation or circuit-psi for unbalanced sets from homomorphic encryption. In: ACM AsiaCCS. pp. 342–356 (2023)
38. Song, X., Yin, D., Bai, J., Dong, C., Chang, E.C.: Secret-shared shuffle with malicious security (2024)
39. Tu, B., Chen, Y., Liu, Q., Zhang, C.: Fast unbalanced private set union from fully homomorphic encryption. In: ACM CCS. pp. 2959–2973 (2023)

40. Tu, B., Zhang, X., Bai, Y., Chen, Y.: Fast unbalanced private computing on (labeled) set intersection with cardinality. Cryptology ePrint Archive, 2023/1015 (2023)
41. Wu, M., Yuen, T.H.: Efficient unbalanced private set intersection cardinality and user-friendly privacy-preserving contact tracing. In: USENIX Security (2023)
42. Yang, Y., Weng, J., Yi, Y., Dong, C., Zhang, L.Y., Zhou, J.: Predicate private set intersection with linear complexity. In: ACNS. Springer (2023)
43. Yao, A.C.: Protocols for secure computations. In: 23rd annual symposium on foundations of computer science (sfcs 1982). IEEE (1982)

Appendix A Ideal Functionalities

Functionality $\mathcal{F}_{\mathrm{OT}}$

Inputs: $\mathcal{F}_{\mathrm{OT}}$ gets two messages (m_0, m_1) from $\mathcal{S}$ and a choice bit $b = \{0, 1\}$ from $\mathcal{R}$.
Functionality: $\mathcal{F}_{\mathrm{OT}}$ outputs the m_b to $\mathcal{R}$ but nothing to $\mathcal{S}$.

Fig. 8. Ideal functionality $\mathcal{F}_{\mathrm{OT}}$ of oblivious transfer

Functionality Π_{ST}

Public Params: A finite field $\mathbb{F}$, the length of ST n.
Functionality: Upon receiving a permutation π over $[n]$ from $\mathcal{R}$,

1) $\mathcal{F}_{\mathrm{ST}}$ samples a vector $\boldsymbol{a} \xleftarrow{\$} \mathbb{F}^n$ and computes the shares $\{\langle a_{\pi(i)}\rangle_1\}_{i\in[n]}$ and $\{\langle a_{\pi(i)}\rangle_0\}_{i\in[n]}$.
2) $\mathcal{F}_{\mathrm{ST}}$ outputs $\boldsymbol{a} = \{a_i\}_{i\in[n]}$, $\{\langle a_{\pi(i)}\rangle_1\}_{i\in[n]}$ to $\mathcal{S}$ and $\{\langle a_{\pi(i)}\rangle_0\}_{i\in[n]}$ to $\mathcal{R}$.

Fig. 9. Ideal functionality $\mathcal{F}_{\mathrm{ST}}$ of share translation

Appendix B Performance comparisons of uPCSI-card-sum

Appendix C Security Proof

Proof. The simulators Sim_R and Sim_S are as follows.
Corrupt $\mathcal{R}$: $Sim_R(Y, \boldsymbol{e}, \langle \boldsymbol{r}\rangle_0)$ first interacts with the trusted third party (TTP) and gets the intersection set I. Then, it invokes the $Sim^R_{\mathrm{pOPRF}}(Y, Y')$ and appends the output to the view. At last, Sim_R encrypts $\{z_i\}_{i\in[n_y]}, \{\langle r_i\rangle_0\}_{i\in[n_y]}$ to simulate the ciphertexts in online step 3. Specifically, $z_i = 0$, if $y_i \in I$ $(e_i = 1)$; otherwise $z_i \xleftarrow{\$} \mathbb{F}$. According to the security of pOPRF, Y' from Sim^R_{pOPRF}

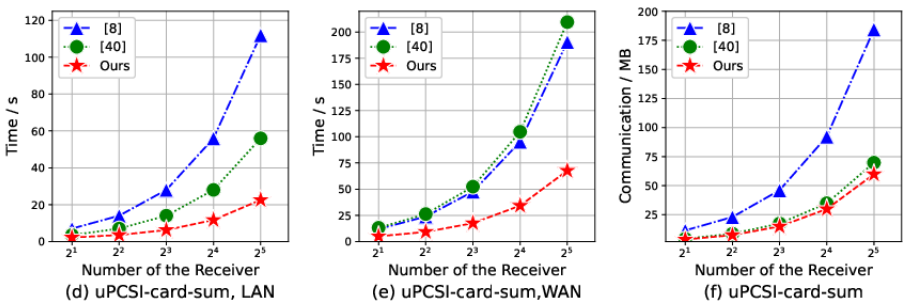

Fig. 10. Total cost (including online and offline phase) comparison among our uPCSI and SOTA works [8, 40] with $n_x = 2^{16}$ and $n_y = 1024$ in the multi-receiver setting.

is pseudo-random and independent of Y. With the circuit privacy of FHE, we guarantee the security of ciphertexts $[\![z_i]\!]_{i\in[n_y]}, [\![\langle r_i\rangle_0]\!]_{i\in[n_y]}$. So, the simulated view is indistinguishable from the real view.

Corrupt $\mathcal{S}$: $Sim_S(X, \langle \boldsymbol{r}\rangle_1)$ first computes the pOPRF results X' with a randomly selected PRF key k. Then, Sim_S invokes the $Sim^R_{\text{pOPRF}}(k)$ and appends the output to the view. Third, it computes coefficients as real protocol. Sim_S samples items $Y' = \{y'_i\}_{i\in[n_y]}$, where $|X \cap Y|$ items from the set X' and others are sampled from the field $\mathbb{F}$. After that, it encrypts $\{y'_i\}_{i\in[n_y]}$ to simulate the ciphertexts in online step 2. The security of the pOPRF and FHE guarantees that the view in simulation is indistinguishable from the view in the real protocol. □

Proof of Lemma 2. We show the simulators Sim_R and Sim_S for uPCSI-card-sum as follows.

Corrupt $\mathcal{S}$: $Sim_S(X)$ chooses a random permutation π, invokes $Sim^S_{\text{pmqPMT}}(\pi)$ and appends the output to the view. Then, Sim_S invokes $Sim^S_{\text{ST}}(\pi)$ and appends output to the view. Sim_S samples random values in place of $\{y'_i\}_{i\in[n_y]}$ in step 3 and generates (m^0_i, m^1_i) as real protocol. Finally, Sim_S invokes $Sim^S_{\text{OT}}(m^0_i, m^1_i)$ and appends output to the view. Since the security of pmqPMT, ST, and OT, the simulation is indistinguishable from real view.

Corrupt $\mathcal{R}$: $Sim_R(Y, k, sum)$ chooses a random vector $\{e_i\}_{i\in[n_y]} \in \{0,1\}^{n_y}$ subject to $\sum_{i=0}^{n_y} e_i = k$, invokes $Sim^R_{\text{pmqPMT}}(Y, \{e_i\}_{i\in[n_y]})$ and appends the output to the view. Then, Sim_R samples random items $\{\langle a_i\rangle_0\}_{i\in[n_y]}$ and $\{a_i\}_{i\in[n_y]}$, invokes $Sim^S_{\text{ST}}(\{a_i\}_{i\in[n_y]}, \{\langle a_i\rangle_0\}_{i\in[n_y]})$ and appends the output to the view. Next, Sim_R samples random values $\{v_i\}_{i\in[n_y]}$ such that $\sum_{i=0}^{n_y} v_i = sum - \sum_{i=1}^{n_y} e_i\langle a_i\rangle_0$ to simulate $\{y'_i\}_{i\in[n_y]}$ in step 3. Finally, Sim_R invokes $Sim^R_{\text{OT}}(\{e_i\}_{i\in[n_y]}, \{v_i\}_{i\in[n_y]})$ and appends the output to the view. The security of pmqPMT, ST, and OT guarantees the view in simulation is indistinguishable from the real view.

For uPCSI-secret-sharing, we exhibit simulators Sim_R and Sim_S as follows.

Corrupt $\mathcal{S}$: $Sim_S(X, \{\langle z_i\rangle_1\}_{i\in[k]})$ chooses a random permutation π, invokes $Sim^S_{\text{pmqPMT}}(\pi, X)$ and appends output to the view. Then, Sim_S samples random items $\{\langle a_i\rangle_1\}_{i\in[n_y]}$, invokes $Sim^S_{\text{ST}}(\pi, \{\langle a_i\rangle_1\}_{i\in[n_y]})$ and appends the output to

the view. Then, Sim_S samples random items $\{\langle b_i\rangle_1\}_{i\in[n_y]}$ and $\{b_i\}_{i\in[n_y]}$, invokes $Sim^S_{\mathrm{ST}}(\{b_i\}_{i\in[n_y]}, \{\langle b_i\rangle_1\}_{i\in[n_y]})$ and appends the output to the view. In step 3, Sim_S chooses a random vector $\mathbf{e} = \{e^*_i\}_{i\in[n_y]}$ subject to $\sum_{i=0}^{n_y} e^*_i = k$ to simulate $\{e^*_i\}_{i\in[n_y]}$ and sample random numbers to simulate $\{y'_i\}_{i\in[n_y]}$. Since the security of pmqPMT and ST, the simulated view is indistinguishable from the real view.

Corrupt $\mathcal{R}$: $Sim_R(Y, \{\langle z_i\rangle_0\}_{i\in[k]})$ samples a vector $\{e_i\}_{i\in[k]}$, then invokes the $Sim^R_{\mathrm{pmqPMT}}(Y, \{e_i\}_{i\in[k]})$ and appends the output to the view. Then, Sim_R samples random $\{\langle a_i\rangle_0\}_{i\in[n_y]}$ and $\{a_i\}_{i\in[n_y]}$, invokes $Sim^S_{\mathrm{ST}}(\{a_i\}_{i\in[n_y]}, \{\langle a_i\rangle_0\}_{i\in[n_y]})$ and appends the output to the view. Next, Sim_R samples a random permutation π^* and random items $\{\langle b_i\rangle_0\}_{i\in[n_y]}$, invokes $Sim^S_{\mathrm{ST}}(\pi^*, \{\langle b_i\rangle_0\}_{i\in[n_y]})$ and appends the output to the view. In step 4, Sim_R samples random numbers to simulate $\{y^*_i\}_{i\in[n_y]}$. The security of pmqPMT and ST guarantees the view in simulation is indistinguishable from the real view. □

Proof of Lemma 4. We exhibit simulators Sim_R and Sim_S as follows.

Corrupt $\mathcal{S}$: $Sim_S(X, \{\langle u_i\rangle_1\}_{i\in[k]}, \{\langle v_i\rangle_1\}_{i\in[k]})$ samples a permutation π and a vector $\{r_i\}_{i\in[n_y]}$, invokes $Sim^S_{\mathrm{pmqPMTwL}}(X, \pi, \{r_i\}_{i\in[n_y]})$ and appends the output to the view. Then, Sim_S invokes $Sim^S_{\text{SL-uPCSI-SS}}(\pi, \{\langle v_i\rangle_1\}_{i\in[k]})$, and appends the output to the view. After that, Sim_S encrypts $\{\langle u_i\rangle_1\}_{i\in[k]}$ to simulate the ciphertexts $\{[\![c_j^{r,\prime}]\!]\}_{j\in[k]}$ in step 3. The security of AHE, the pmqPMT with labels and uPCSI-secret-sharing protocol guarantees the view in simulation is indistinguishable from the real view.

Corrupt $\mathcal{R}$: $Sim_R(Y, \{\langle u_i\rangle_0\}_{i\in[k]}, \{\langle v_i\rangle_0\}_{i\in[k]})$ samples random vectors $\{e_i\}_{i\in[n_y]}$, $\{r_i\}_{i\in[n_y]}$, invokes $Sim^R_{\mathrm{pmqPMTwL}}(Y, \{e_i\}_{i\in[n_y]}, \{r_i\}_{i\in[n_y]})$ and appends the output to the view. Then, Sim_R samples a random permutation π^* and a random vector $\boldsymbol{v}^0$, invokes $Sim^R_{\text{uPCSI-SS}}(\pi^*, \{\langle v_i\rangle_0\}_{i\in[k]})$, and appends the output to the view. After that, Sim_R encrypts random values to simulate the ciphertexts in step 3. The security of AHE, the pmqPMT with labels and uPCSI-secret-sharing protocol guarantees the view in simulation is indistinguishable from the real view. □